

ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS

ENSAM MEKNÈS — Maroc

Filière IATD SI — Intelligence Artificielle et Technologies des Données : Systèmes Industriels
Section 8

RAPPORT DE TRAVAUX PRATIQUE 2

Création d'une Salle Virtuelle en Unity 6

*Application VR pour Meta Quest 2
2ème année — Cycle Ingénieur*

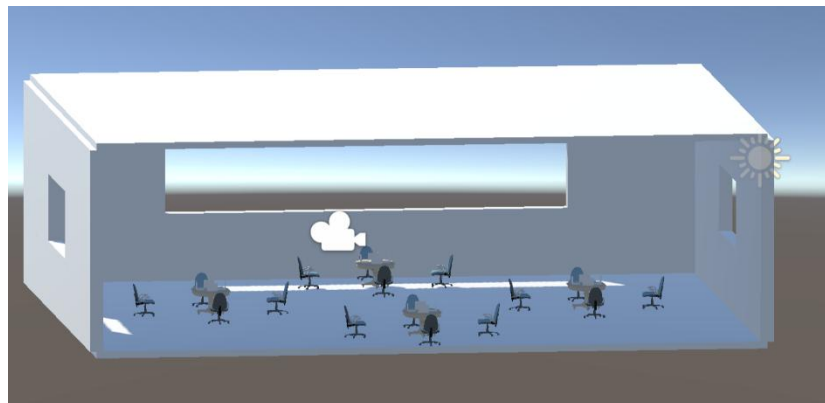
Par :

DJERI ALASSANI OUBENOUPOU
NANKOULI MARC THIERRY

Encadrant : Mr Ajebli Ahmed | Maalouf Imad

Element : Virtual and Augmented Reality

Date : 18/04/26



Introduction

Ce rapport présente l'ensemble des manipulations effectuées lors du travail pratique consacré à la réalité virtuelle. L'objectif principal de ce projet consistait à concevoir et à construire une salle virtuelle interactive sous le moteur de jeu Unity 6, destinée à être déployée et explorée de manière naturelle à l'aide d'un casque Meta Quest 2.

Domaine en pleine expansion, la réalité virtuelle nécessite la mobilisation de compétences variées, allant de la modélisation 3D à la programmation, en passant par la conception d'interfaces et l'application des lois de la physique. Concrètement, notre vision pour ce TP était de concevoir un environnement crédible et immersif comprenant :

- Un espace fermé, délimité par un sol, un plafond et quatre murs ;
- Plusieurs tables disposées dans la pièce, accompagnées de chaises ;
- Des ordinateurs portables posés sur les espaces de travail ;
- Des objets capables de réagir correctement et de manière réaliste aux lois de la physique.

À travers la réalisation de cette salle, nous avons été amenés à manipuler et à mettre en pratique plusieurs notions fondamentales du développement VR. Cela inclut la configuration d'un projet XR, la construction d'un environnement 3D par l'assemblage d'objets simples, l'importation de modèles externes et la création de *Prefabs* réutilisables. Nous avons également intégré la physique via les composants *Collider* et *Rigidbody*, et automatisé certains comportements grâce à des scripts C#.

Le présent document décrit chaque étape de la réalisation de manière détaillée, justifie les choix techniques effectués tout au long du développement et explique l'utilité de chaque action dans la logique globale du projet.

1. Vision du projet

1.1. Le problème à résoudre

Dans tout projet de réalité virtuelle, l'approche consistant à placer manuellement chaque objet dans la scène montre très rapidement ses limites. Plusieurs problèmes apparaissent :

- les dimensions deviennent incohérentes à l'usage ;
- les objets peuvent se traverser lorsqu'il manque des Colliders ;
- les objets mobiles restent immobiles en l'absence de Rigidbody ;
- le placement manuel devient long et fastidieux dès qu'une modification de la salle est nécessaire.

Le TP poursuivait donc un double objectif : d'une part, construire correctement la scène sous Unity en respectant les règles de dimensionnement et de physique ; d'autre part, automatiser une partie du travail par l'intermédiaire de scripts C# afin de gagner en efficacité et en cohérence.

1.2. Chronologie du TP

Le travail s'est déroulé selon la progression suivante :

Étape	But pédagogique
1. Préparation du projet	Créer ou ouvrir un projet Unity 6 configuré pour la VR.
2. Organisation de la scène	Repérer l'interface Unity et organiser les dossiers du projet.
3. Construction de la salle	Utiliser des Cubes et modifier leurs dimensions.

Étape	But pédagogique
4. Import des modèles 3D	Intégrer les tables, chaises et ordinateurs portables.
5. Création des Prefabs	Réutiliser les mêmes objets sans reconfiguration.
6. Physique	Ajouter Colliders, Rigidbody et gravité aux objets utiles.
7. Scripts C#	Ajouter des Components et automatiser le placement.
8. Test VR	Vérifier le comportement dans l'éditeur puis sur casque.

Cette progression n'est pas arbitraire : elle suit une logique pédagogique précise. La salle est d'abord construite manuellement afin de bien assimiler les notions de dimensions et d'unités. Ce n'est qu'ensuite que le mobilier est placé automatiquement, ce qui permet de gagner du temps tout en conservant une scène propre et modifiable.

2. Préparation du projet Unity 6

2.1. Prérequis

Avant d'entamer les manipulations, il a été nécessaire de vérifier la présence et la bonne installation des éléments suivants :

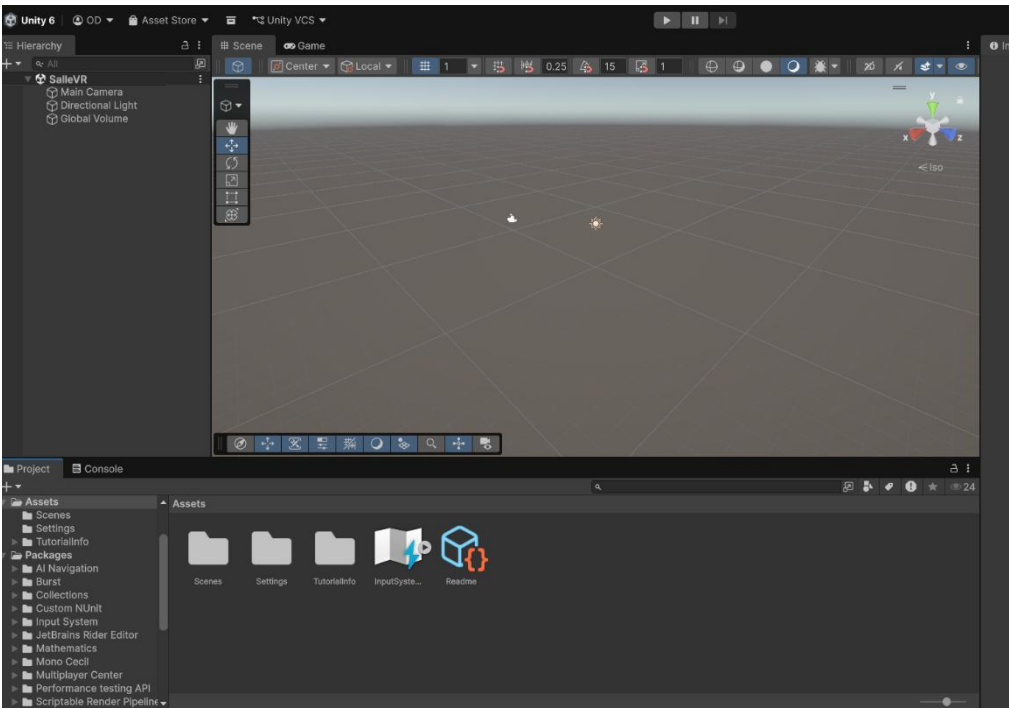
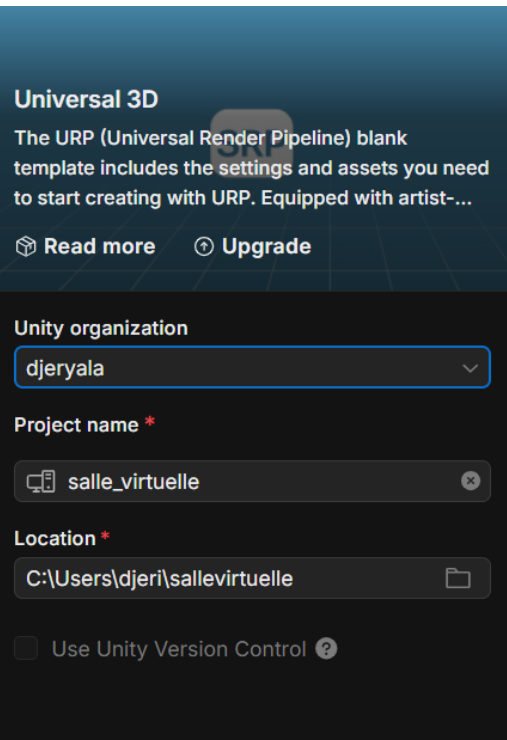
- Unity 6 installé via Unity Hub ;
- le module Android Build Support ajouté à Unity Hub, indispensable pour le déploiement sur Meta Quest 2 qui fonctionne sous Android ;
- un projet créé avec le VR Template, ou à défaut un projet 3D dans lequel les packages OpenXR et XR Interaction Toolkit ont été ajoutés manuellement ;
- un casque Meta Quest 2 pour les tests finaux ;
- les modèles 3D représentant la table, la chaise et l'ordinateur portable.

Ces prérequis sont essentiels : sans le module Android, le build vers le casque est impossible ; sans OpenXR, aucune interaction VR ne peut être gérée.

Création du projet

Les étapes de création du projet ont été les suivantes :

- 1. Ouverture de Unity Hub.
- 2. Clic sur New Project.
- 3. Sélection du VR Template proposé par Unity 6.
- 4. Attribution d'un nom au projet puis clic sur Create Project.



Création du projet Unity 6 avec le 3D VR Template

Le choix du VR Template constitue la solution la plus adaptée dans un contexte de TP, car il prépare automatiquement les briques techniques essentielles :

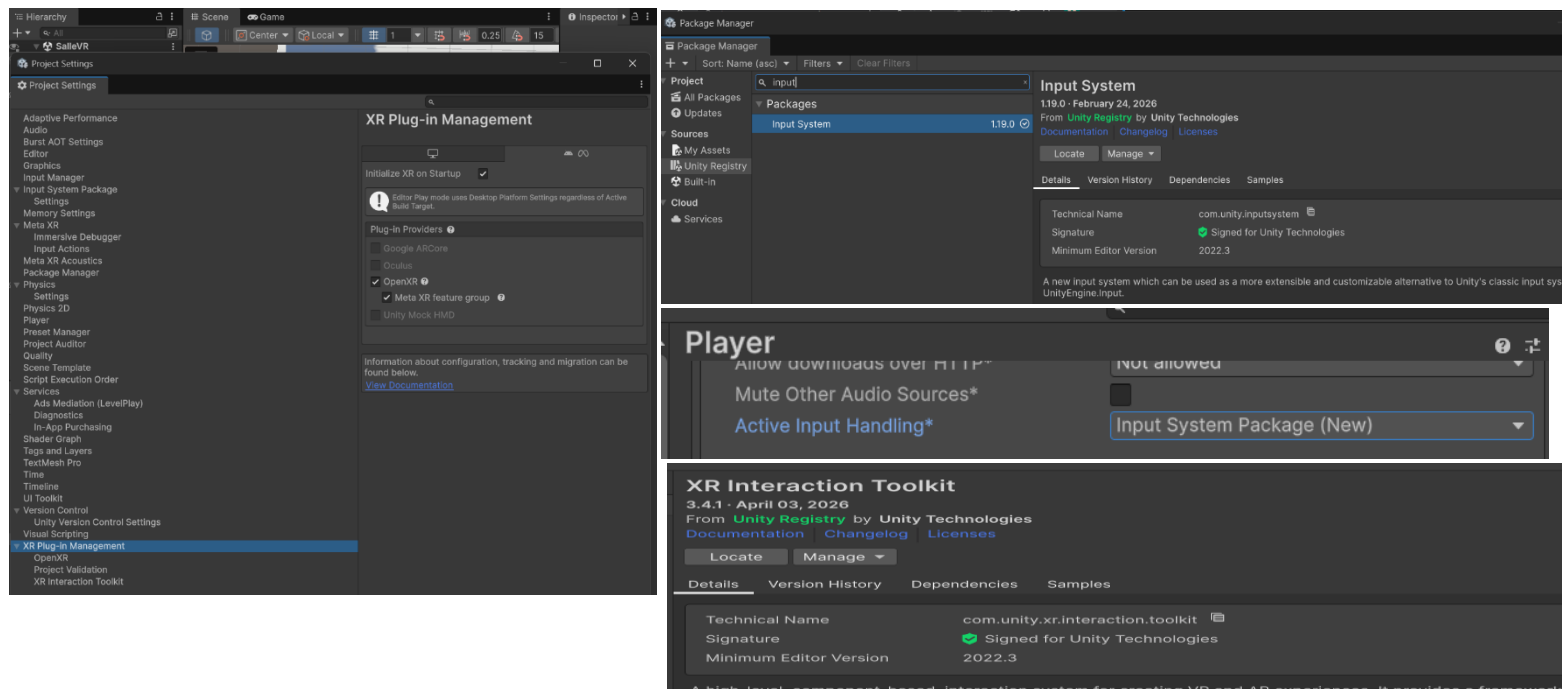
- l'URP (Universal Render Pipeline), qui optimise le rendu graphique sur les plateformes mobiles comme le Quest ;
- le plug-in OpenXR, standard ouvert de gestion des appareils VR ;
- le XR Interaction Toolkit, qui fournit les composants d'interaction avec les manettes ;
- une base de scène XR pré-configurée, incluant notamment un XR Origin.

Utiliser ce template nous a évité de passer par une configuration manuelle longue et source d'erreurs.

2.2. Vérification de la configuration XR

Dans le cas où le projet n'aurait pas été créé à partir du VR Template, il aurait été nécessaire de vérifier manuellement la configuration XR. Les étapes sont les suivantes :

- **1.** Accéder à Edit → Project Settings → XR Plug-in Management.
- **2.** Activer OpenXR pour la plateforme Android.
- **3.** Vérifier que le package Input System est actif.
- **4.** Installer XR Interaction Toolkit via le Package Manager si besoin.



Fenêtre XR Plug-in Management avec OpenXR activé et fenêtre du package Input System actif

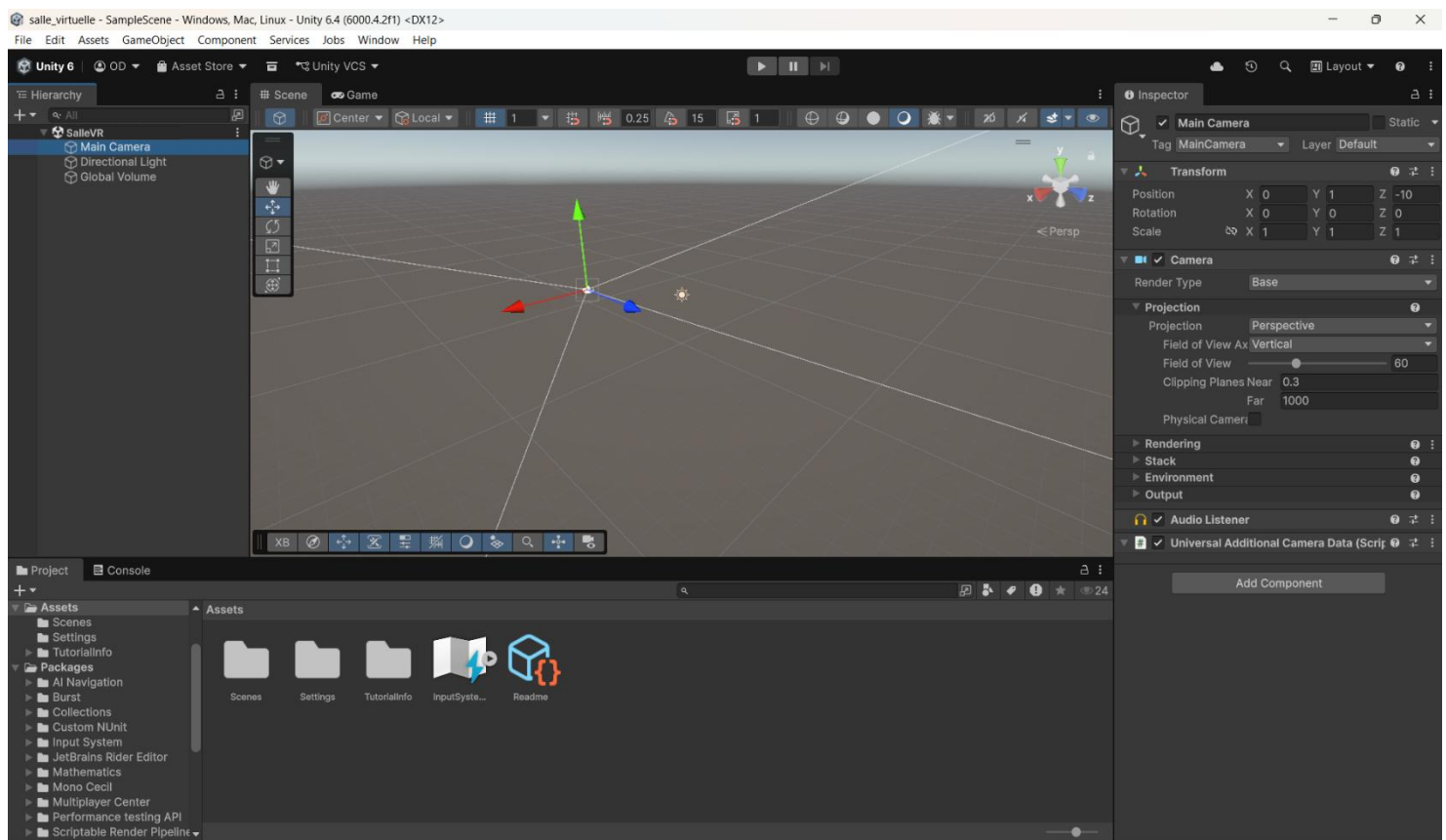
Cette vérification est indispensable : un projet VR sous Unity 6 repose sur la combinaison XR Origin, OpenXR et XR Interaction Toolkit. Cette triade permet de gérer la caméra du casque, les manettes et les interactions entre l'utilisateur et la scène. Sans cette configuration, le casque ne peut tout simplement pas afficher la scène.

3. Prise en main de l'interface Unity

Avant toute manipulation, nous avons identifié les quatre zones de l'interface Unity que nous allons utiliser de manière permanente tout au long du TP :

Zone	Rôle
Hierarchy	Liste de tous les objets présents dans la scène.
Scene	Vue 3D pour construire et déplacer les objets.
Inspector	Panneau de réglage de l'objet actuellement sélectionné.
Project	Dossiers du projet (scripts, prefabs, modèles, scènes).

Une règle fondamentale est à retenir dès cette étape : dans Unity, 1 unité = 1 mètre. Cette correspondance est particulièrement critique en VR, car toute incohérence d'échelle est immédiatement perçue par l'utilisateur. Une salle trop petite paraît oppressante, une salle trop grande paraît vide et irréaliste. Respecter cette règle dès la conception évite des corrections longues par la suite.

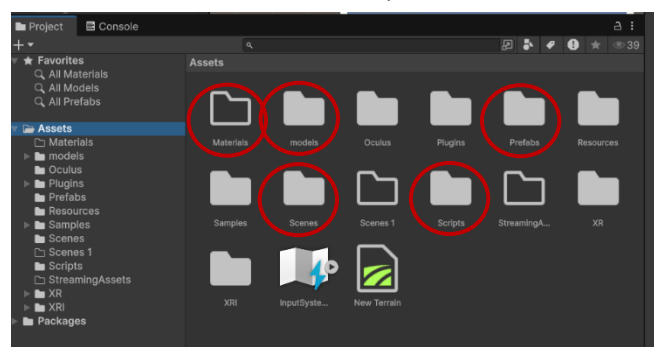


Interface Unity avec les quatre zones de travail

3.1. Organisation des dossiers du projet

Nous avons ensuite créé une arborescence de dossiers dans le panneau Project afin de maintenir un projet propre et organisé. L'organisation des fichiers est un élément trop souvent négligé, mais elle conditionne la maintenabilité du projet.

- 1. Clic droit sur Assets dans le panneau Project.
- 2. Sélection de Create → Folder.
- 3. Création des dossiers : Scenes, Models, Prefabs, Scripts, Materials.



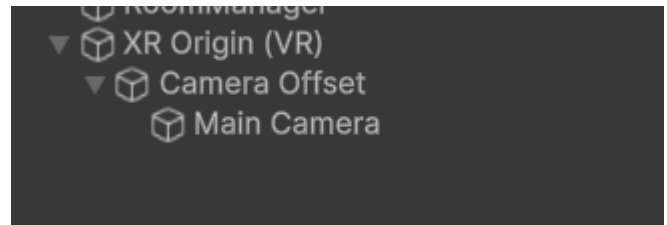
Arborescence des dossiers dans le panneau Project

Chaque dossier remplit une fonction claire : Scenes accueille les scènes Unity, Models contient les fichiers 3D importés, Prefabs regroupe les objets réutilisables, Scripts centralise le code C#, et Materials contient les matériaux et textures. Cette structuration permet de retrouver rapidement les ressources et d'éviter un mélange anarchique des fichiers.

4. Mise en place de la scène VR

- un XR Origin, qui représente la position de l'utilisateur dans l'espace virtuel ;
- une caméra gérée par le casque, associée au XR Origin ;
- une lumière principale, pour que les objets soient visibles ;
- un environnement 3D, c'est-à-dire les objets de la scène proprement dits.

- Inspection de la Hierarchy pour repérer les objets présents.
- Vérification qu'un XR Origin était déjà présent (ce qui était le cas grâce au VR Template).
- **Le problème** était que le XR origin qui était présent ne contenait pas de contrôleur (nécessaire pour manipuler le **Meta Quest** plus tard mais juste un camera :



Aller dans **Projet>Assets>Samples>XR Interaction Setup >3.4.1>Starter Assets> Prefabs** et choisir ce XR origin :



5. Construction de la salle avec des Cubes

5.1. Principe général

Unity ne fournit pas d'objet « mur » ou « sol » prêt à l'emploi. La démarche consiste donc à utiliser des objets primitifs de type Cube, puis à modifier leur échelle dans l'Inspector pour leur donner les dimensions voulues. Cette approche est simple, maîtrisable et permet de comprendre concrètement comment un espace 3D est construit.

Les dimensions cibles de notre salle sont fixées à 10 mètres sur l'axe X pour la largeur ; 8 mètres sur l'axe Z pour la profondeur ; 3 mètres sur l'axe Y pour la hauteur .

5.2. Création du sol , des quatre murs et du plafond

L'architecture de la salle repose sur les dimensions requises pour ce TP : 10 mètres de largeur (axe X), 8 mètres de profondeur (axe Z) et 3 mètres de hauteur sous plafond (axe Y). Afin de garantir une physique réaliste en réalité virtuelle (VR), l'échelle standard d'Unity (1 unité = 1 mètre) a été scrupuleusement respectée.

Pour faciliter la gestion de la scène, chaque façade est regroupée dans un objet parent vide (Empty GameObject) placé à la position (0, 0, 0) avec une échelle de (1, 1, 1). Cela permet d'utiliser les coordonnées globales de la salle pour placer les sous-éléments (cubes) formant les murs, les fenêtres et la porte.

Tableau récapitulatif de tous les éléments structurels

Objet Parent	Cube Enfant	Pos X	Pos Y	Pos Z	Scale X	Scale Y	Scale Z	Fonction
Floor	-	0	-0.05	0	10	0.1	8	Sol de la salle
Ceiling	-	0	3.05	0	10	0.1	8	Plafond de la salle
Wall_North	-	0	0	0	1	1	1	Conteneur (Empty) Empty object
Wall_North	Left_Wall	-3	1.5	4	4	3	0.2	Mur gauche de l'entrée
Wall_North	Right_Wall	3	1.5	4	4	3	0.2	Mur droit de l'entrée
Wall_North	Top_Wall	0	2.5	4	2	1	0.2	Linteau de la porte
Wall_North	Door_North	0	1	4	1.98	1.98	0.2	Porte VR fonctionnelle
Wall_South	-	0	0	0	1	1	1	Conteneur (Empty) Empty object
Wall_South	Bottom	0	0.5	-4	10	1	0.2	Allège sous la fenêtre
Wall_South	Top	0	2.5	-4	10	1	0.2	Haut du mur
Wall_South	Left_Side	-3	1.5	-4	4	1	0.2	Montant gauche
Wall_South	Right_Side	3	1.5	-4	4	1	0.2	Montant droit
Wall_East	-	0	0	0	1	1	1	Conteneur (Empty) Empty object
Wall_East	Bottom	5	0.5	0	0.2	1	8	Allège sous la fenêtre
Wall_East	Top	5	2.5	0	0.2	1	8	Haut du mur

Objet Parent	Cube Enfant	Pos X	Pos Y	Pos Z	Scale X	Scale Y	Scale Z	Fonction
Wall_East	North_Side	5	1.5	2.5	0.2	1	3	Montant côté Nord
Wall_East	South_Side	5	1.5	-2.5	0.2	1	3	Montant côté Sud
Wall_West	-	0	0	0	1	1	1	Conteneur (Empty) Empty object
Wall_West	Bottom	-5	0.5	0	0.2	1	8	Allège sous la fenêtre
Wall_West	Top	-5	2.5	0	0.2	1	8	Haut du mur
Wall_West	North_Side	-5	1.5	2.5	0.2	1	3	Montant côté Nord
Wall_West	South_Side	-5	1.5	-2.5	0.2	1	3	Montant côté Sud

Justifications des choix dimensionnels

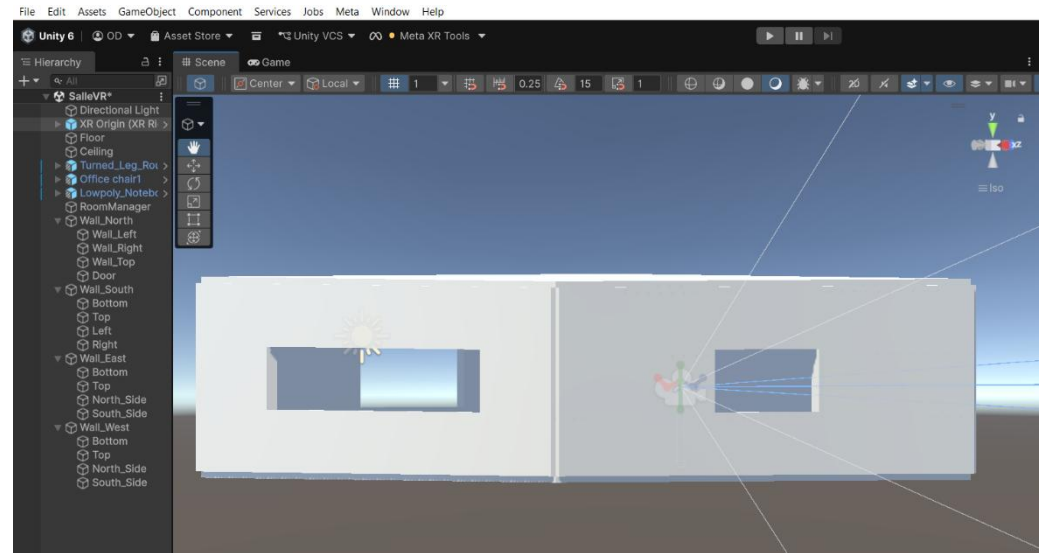
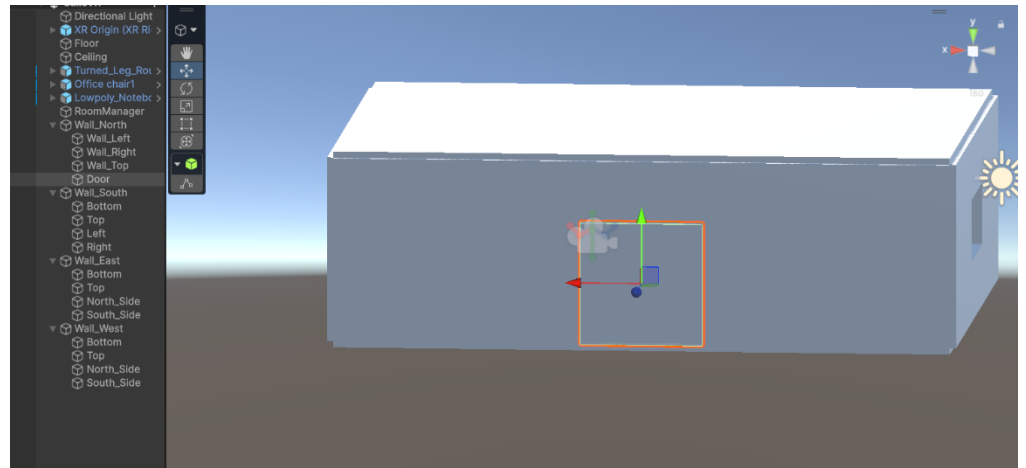
Justification de l'axe Y pour le sol et le plafond : Un Cube dans Unity est centré sur son pivot (son milieu). Le sol ayant une épaisseur (Scale Y) de 0.1 (soit 10 cm), le positionner à Y = 0 l'enfoncerait à moitié. Le placer à Y = -0.05 permet à la surface supérieure du sol d'être exactement au niveau 0, ce qui est indispensable pour le calibrage de la hauteur du joueur en VR. De même, le plafond est placé à 3.05 pour garantir une hauteur libre exacte de 3 mètres à l'intérieur de la pièce.

Épaisseur des Murs (Scale 0.2) : L'épaisseur des murs a été fixée à 0.2 mètres (20 cm). Cela donne un rendu visuel plus réaliste et robuste en VR qu'une épaisseur de 0.1, et prévient les erreurs de calcul du moteur physique (passer à travers le mur) lors des déplacements de l'utilisateur.

Logique de Centrage (Pos Y = 1.5) : Pour qu'un mur de 3 mètres de haut pose bien sa base sur le sol (Y = 0), son centre de gravité doit être situé exactement à 1.5 mètre du sol. Cette logique s'applique à tous les sous-éléments (par exemple, les morceaux de murs à côté des fenêtres ayant une hauteur de 1m sont placés à Y = 1.5 pour flotter entre les éléments bas et hauts).

Configuration des murs Nord et Sud (axe Z = ±4) : Les murs Nord (avec la porte) et Sud (avec une fenêtre de 2m × 1m) s'étendent sur 10 mètres de large.

Configuration des murs Est et Ouest (axe X = ±5) : Les murs Est et Ouest ferment la salle sur une profondeur de 8 mètres. Ils comportent chacun une fenêtre centrale de 2m de large sur 1m de haut.



Salle complète avec les quatre murs, le sol et le plafond

5.3. Modification des dimensions de la salle

Si l'on souhaite agrandir ou rétrécir la salle, il ne suffit pas de modifier un seul élément. Trois éléments doivent être ajustés simultanément :

- le Scale du sol et du plafond ;
- la Position des murs, pour qu'ils restent aux extrémités de la pièce ;
- les valeurs Room Width et Room Depth du script RoomGenerator, pour que le placement automatique reste cohérent.

À titre d'exemple, pour une largeur portée à 12 mètres au lieu de 10, il faudrait : passer le Scale X du sol à 12, placer le mur Est à Pos X = 6 et le mur Ouest à Pos X = -6, puis mettre à jour le script en conséquence.

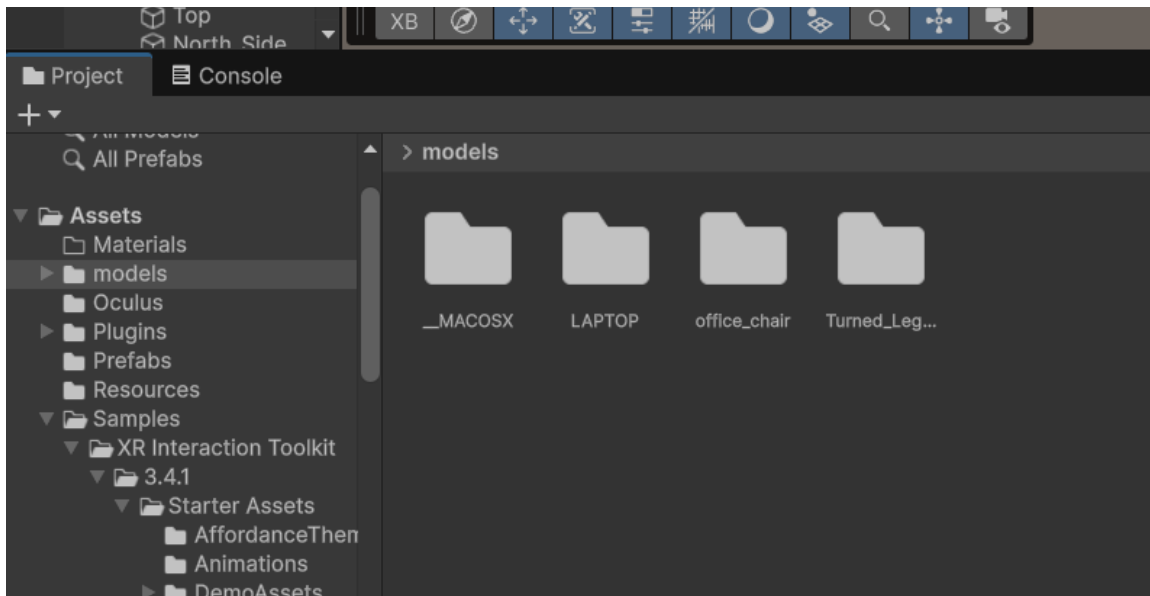
Une erreur fréquente à ce stade consiste à modifier les dimensions des murs sans mettre à jour le script : le placement automatique des tables et chaises se retrouve alors décalé, avec des objets qui apparaissent dans les murs ou à l'extérieur de la salle.

6. Importation des modèles 3D

6.1. Import des fichiers

Pour donner vie à la salle, nous avons importé des modèles 3D représentant les meubles (table, chaise, ordinateur portable). La procédure a été la suivante :

- **Telecharger le zip via le lien drive fourni ;** Ouverture de l'explorateur de fichiers du système d'exploitation.
- Glisser-déposer des fichiers 3D dans le dossier Assets/Models du projet.
- Import simultané du fichier .obj, du fichier .mtl associé et des textures éventuelles.



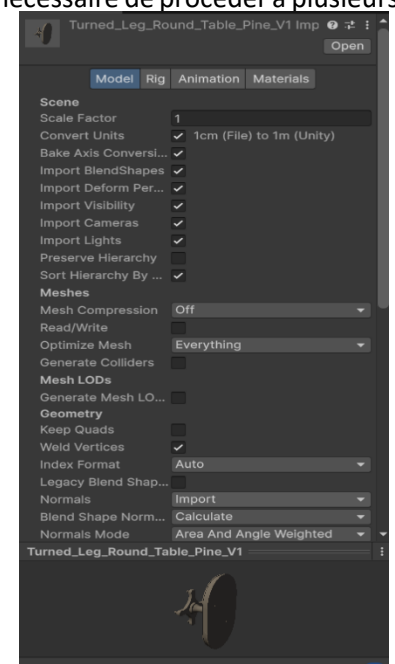
Modèles 3D importés dans le dossier Models

L'import conjoint des trois types de fichiers est important : le fichier .obj contient la géométrie du modèle, le fichier .mtl décrit les matériaux utilisés, et les textures fournissent l'apparence visuelle. L'absence de l'un de ces éléments conduit à un objet sans texture ou avec un rendu incorrect.

6.2. Réglage de l'échelle et de l'orientation

Les modèles 3D ne sont presque jamais utilisables tels quels après importation. Il est nécessaire de procéder à plusieurs vérifications pour chacun d'eux (Laptop ; table ; chaise) :

- **1.** Sélection du modèle dans le panneau Project.
- **2.** Ouverture de l'onglet Model dans l'Inspector.
- **3.** Vérification du Scale Factor (facteur d'échelle).
- **4.** Activation de Bake Axis Conversion si l'objet apparaît couché.
- **5.** Clic sur Apply pour appliquer les modifications.



Paramètres d'import du modèle dans l'Inspector

Trois problèmes classiques peuvent survenir après import et nous les avons effectivement rencontrés :

- objet couché : il s'agit d'une divergence d'axes entre le logiciel d'origine et Unity. La solution consiste à activer Bake Axis Conversion ou à appliquer une rotation manuelle ;
- objet disproportionné : le modèle a été créé avec une unité différente (centimètres, pouces). Le Scale Factor doit alors être ajusté pour retrouver la bonne échelle ;
- objet sans texture : les fichiers de matériaux ou de textures .mlt sont manquants ou n'ont pas été importés dans le même dossier que le fichier .obj.

Ces corrections sont indispensables car un objet mal dimensionné ou mal orienté devient inexploitable en VR, où la sensation d'échelle est primordiale.

6.3. Implémentation de l'interaction physique sur la porte

Pour renforcer l'immersion dans l'environnement virtuel, la porte d'accès à la salle a été configurée pour être manipulable physiquement par l'utilisateur. Contrairement aux équipements intérieurs (tables, chaises, ordinateurs) qui sont gérés et configurés automatiquement par le script RoomGenerator.cs, la porte est un élément de l'architecture nécessitant une configuration manuelle spécifique.

L'objectif n'était pas de déclencher une simple animation d'ouverture automatique au passage du joueur, mais d'offrir une véritable interaction mécanique. Pour ce faire, quatre composants ont été ajoutés au GameObject Door :

Box Collider : Il permet à la porte d'avoir une présence solide dans le monde physique. Le composant ayant été initialisé avec une taille par défaut de (1, 1, 1), ses dimensions se sont mathématiquement multipliées avec le Scale de l'objet défini dans le Transform (X: 1.98, Y: 1.98, Z: 0.2). Ainsi, la boîte de collision englobe parfaitement la géométrie 3D de la porte de manière native.

XR Grab Interactable : Ce composant du XR Interaction Toolkit indique au moteur que l'objet peut être ciblé et saisi par les manettes de l'utilisateur. Le Movement Type a été configuré en Velocity Tracking afin que la force exercée par le joueur soit transmise de manière réaliste au corps physique de la porte, sans téléportation instantanée.

Rigidbody : Il soumet la porte aux lois de la physique d'Unity, réagissant ainsi aux forces appliquées par le joueur lorsqu'il la saisit. Deux réglages ont été validés comme essentiels : Use Gravity est décoché pour éviter que la porte ne tombe sous son propre poids, et Is Kinematic est également décoché pour que le moteur physique puisse lui appliquer des forces. Sans ce dernier réglage, la porte resterait totalement immobile quelle que soit l'interaction.

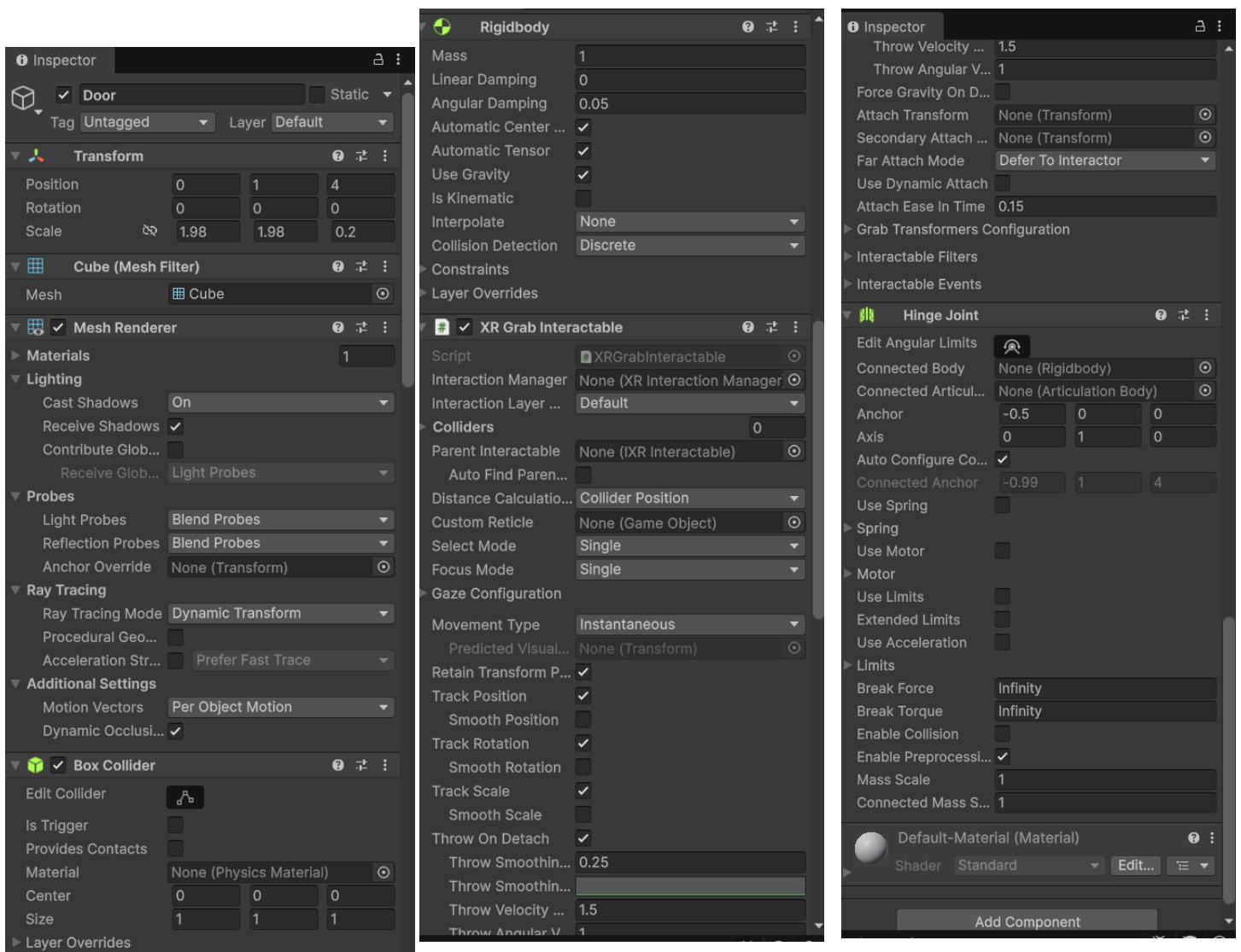
Hinge Joint (Charnière) : C'est le composant central du réalisme de l'interaction. Il lie la porte au mur et restreint ses mouvements à une simple rotation, mimant de vrais gonds. La configuration retenue est la suivante : l'axe de rotation (Axis) est verrouillé sur l'axe Y (0, 1, 0) pour garantir un pivotement purement horizontal, et le point d'ancrage (Anchor) est décalé sur le bord latéral de la porte (X = -0.5) afin que la rotation s'effectue sur le côté, évitant ainsi un pivotement irréaliste par le centre du modèle. La propriété Use Limits a été activée et la section Limits dépliée pour définir un angle d'ouverture de Min = 0° et Max = 90°, empêchant la porte de tourner à 360° comme une toupie. Les valeurs Break Force et Break Torque ont été laissées à Infinity pour que la porte ne se détache jamais de son encadrement sous l'effet des interactions.

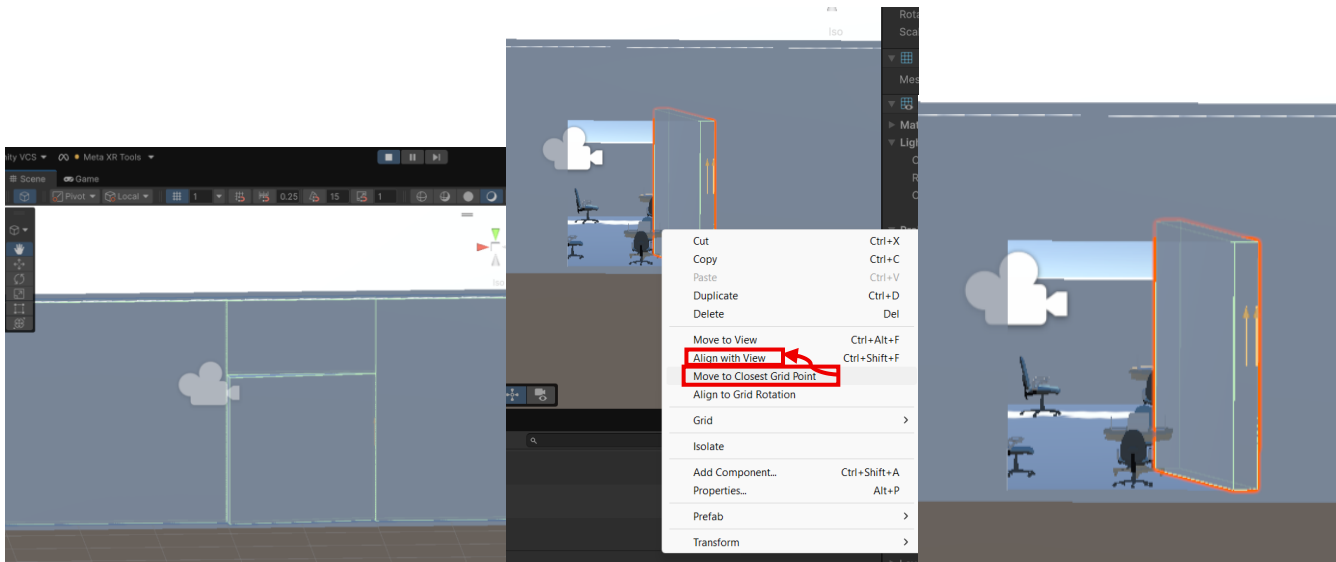
Fonctionnement en simulation et en Réalité Virtuelle :

En ce qui concerne les tests dans l'éditeur Unity, le projet n'intègre pas de XR Device Simulator (le menu Window → XR ne proposait que les options OpenXR et non un simulateur de manettes). De ce fait, il n'est pas possible de simuler une saisie complète directement sur PC. La vérification de la configuration a donc été effectuée statiquement dans

l'Inspector en contrôlant la cohérence de chaque composant, ce qui constitue la méthode de validation appropriée avant un build sur casque.

Avec le casque VR (Meta Quest 2), l'interaction prend tout son sens et devient purement naturelle. L'utilisateur s'approche physiquement de la porte, tend son bras réel vers l'objet et maintient la gâchette latérale (Grip) de la manette pour saisir le battant. En tirant la manette vers lui, le joueur applique une force physique transmise par le XR Grab Interactable. Le Hinge Joint contraint alors cette force à un arc de cercle borné entre 0° et 90° : la porte s'ouvre progressivement sans jamais dépasser l'angle limite ni se détacher de son encadrement. Ce couplage entre l'action physique réelle du joueur et la mécanique virtuelle offre un sentiment de présence et de réalisme particulièrement fort dans l'environnement VR.





7. Création des Prefabs

7.1. Intérêt des Prefabs

Un Prefab est une version sauvegardée et réutilisable d'un objet déjà configuré. Cette notion est centrale dans Unity car elle apporte trois bénéfices majeurs :

- réutiliser le même objet plusieurs fois dans une scène sans le dupliquer manuellement ;
- conserver les mêmes Components et les mêmes réglages sur toutes les copies ;
- éviter de refaire chaque configuration à la main, avec les erreurs et incohérences que cela implique.

De plus, toute modification apportée à un Prefab se répercute automatiquement sur toutes ses instances dans la scène. Cela représente un gain de temps considérable lorsque l'on souhaite, par exemple, modifier la couleur ou la taille de toutes les chaises d'un seul geste.

7.2. Choix entre Box Collider et Mesh Collider

Unity propose plusieurs types de Colliders. Les deux principaux utilisés dans ce TP sont :

Composant	Quand l'utiliser
Box Collider	Forme simple rectangulaire, rapide à calculer. Adapté quand une collision approximative suffit.
Mesh Collider	Forme précise qui épouse la géométrie du modèle. Plus coûteux en ressources.

Pour la chaise, deux approches étaient envisageables :

- l'approche simple, consistant à utiliser un Box Collider : rapide, légère en calcul, mais la collision ne correspond pas exactement à la forme réelle de la chaise ;
- l'approche précise, utilisant un Mesh Collider : la collision épouse fidèlement le modèle 3D, ce qui améliore le réalisme.

Un point technique très important mérite d'être souligné : lorsqu'un objet mobile (doté d'un Rigidbody) utilise un Mesh Collider, il est généralement indispensable d'activer l'option Convex. Sans cette option, Unity refuse certains comportements physiques sur les objets en mouvement, ce qui peut provoquer des erreurs ou des comportements

inattendus. Cette subtilité s'explique par la complexité mathématique qu'implique le calcul de collisions entre deux maillages concaves.

Dans notre cas, nous avons retenu un Box Collider pour la chaise, la forme approximative étant suffisante pour notre usage pédagogique et préservant les performances du casque Meta Quest 2, dont les ressources sont plus limitées que celles d'un PC.

7.3. Création du Prefab de la table

La table est l'objet le plus simple à configurer. La procédure suivie a été :

- 1. Glisser le modèle de table depuis Assets/Models vers la Hierarchy.
- 2. Corriger l'orientation si le modèle apparaissait couché ou mal placé.
- 3. Ajouter un Collider pour que la table puisse supporter d'autres objets.
- 4. Glisser l'objet depuis la Hierarchy vers Assets/Prefabs pour créer le Prefab.
- 5. Renommer le Prefab en TablePrefab.

L'ajout du Collider est essentiel : sans lui, les objets posés sur la table (comme l'ordinateur portable) tomberaient à travers elle. Le Collider fournit la surface de contact invisible que le moteur physique utilise pour détecter les collisions.

7.4. Création du Prefab de la chaise : Collider, Rigidbody et gravité

La chaise est l'objet le plus complet du TP, car elle combine toutes les notions de physique. Elle constitue donc un exemple pédagogique idéal pour comprendre le fonctionnement du moteur physique d'Unity. La procédure appliquée a été :

- 1. Glisser le modèle de chaise dans la scène.
- 2. Ajouter un Rigidbody, composant qui active la physique sur l'objet.
- 3. Vérifier que l'option Use Gravity est bien cochée.
- 4. Laisser l'option Is Kinematic décochée pour que la chaise puisse bouger librement.
- 5. Ajouter un Collider adapté à la forme de la chaise.
- 6. Glisser la chaise vers Assets/Prefabs pour créer ChairPrefab.

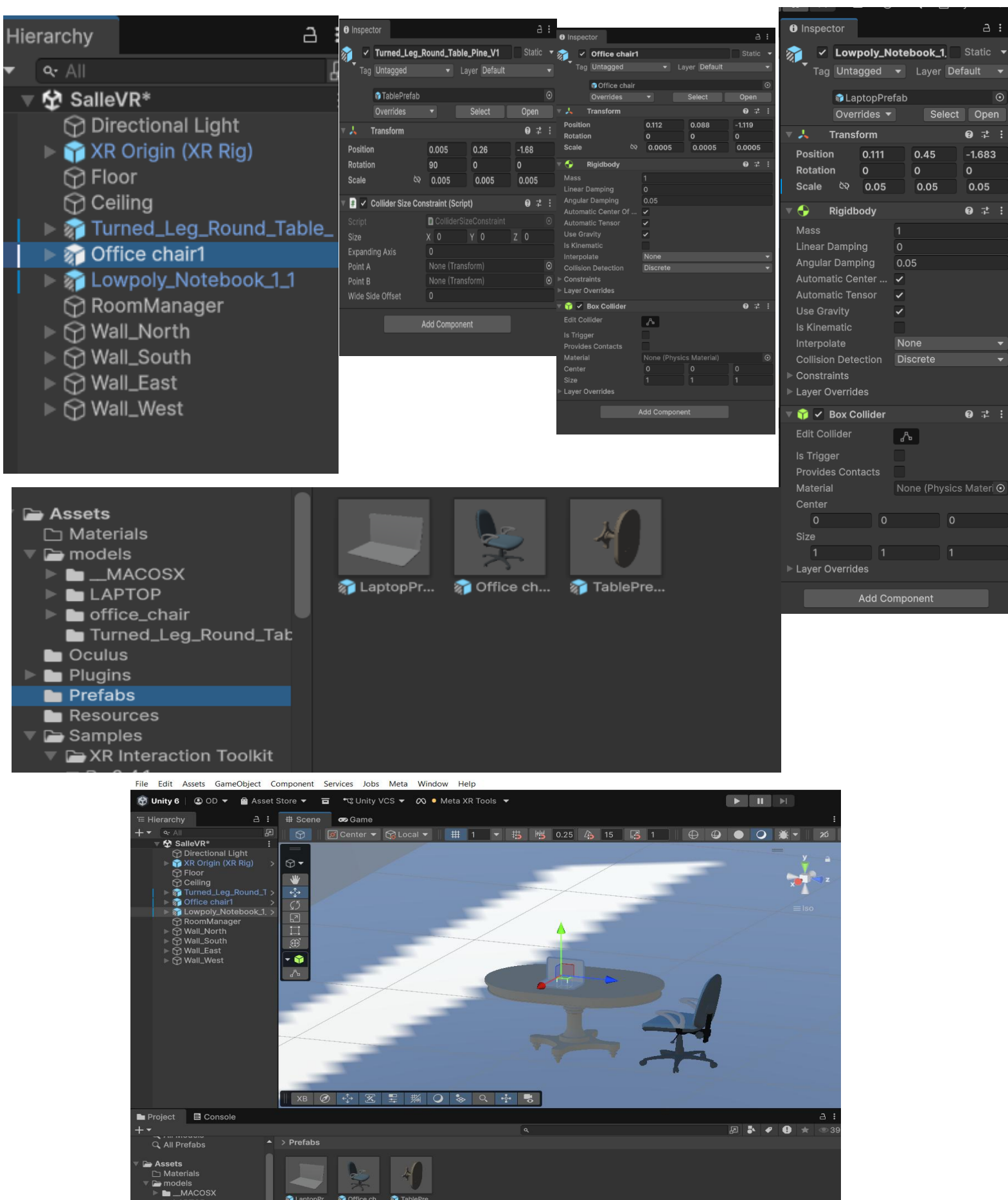
Chacun de ces réglages a une signification précise. Le Rigidbody est le composant qui fait basculer un objet statique vers un objet géré par la physique. Use Gravity active la chute libre sous l'effet de la gravité terrestre (9.81 m/s^2). Is Kinematic, si coché, désactiverait la physique : la chaise ne tomberait plus et ne pourrait être déplacée que par script. Comme nous souhaitons que la chaise se comporte comme un objet réel, cette option a été laissée décochée.

7.5. Cas de l'ordinateur portable

L'ordinateur portable suit la même logique que la chaise. Sa configuration détaillée a été laissée comme exercice d'application. Les étapes attendues étaient :

- choisir un Collider adapté à la forme du laptop ;
- ajouter un Rigidbody si l'objet devait être saisissable par l'utilisateur ;
- vérifier que la gravité est active ;
- transformer le modèle configuré en Prefab (LaptopPrefab).

Pour notre part, nous avons retenu un Box Collider pour l'ordinateur portable, la forme générale étant suffisamment proche d'un parallélépipède pour que cette approximation soit satisfaisante. Le Rigidbody a été ajouté pour permettre la saisie et la manipulation de l'objet en VR par l'utilisateur.



8. Scripts C# et ajout de Components

8.1. Intérêt du passage par des scripts

Jusqu'à ce stade, toutes les manipulations ont été effectuées manuellement dans l'interface d'Unity. L'introduction de scripts C# répond à trois objectifs concrets :

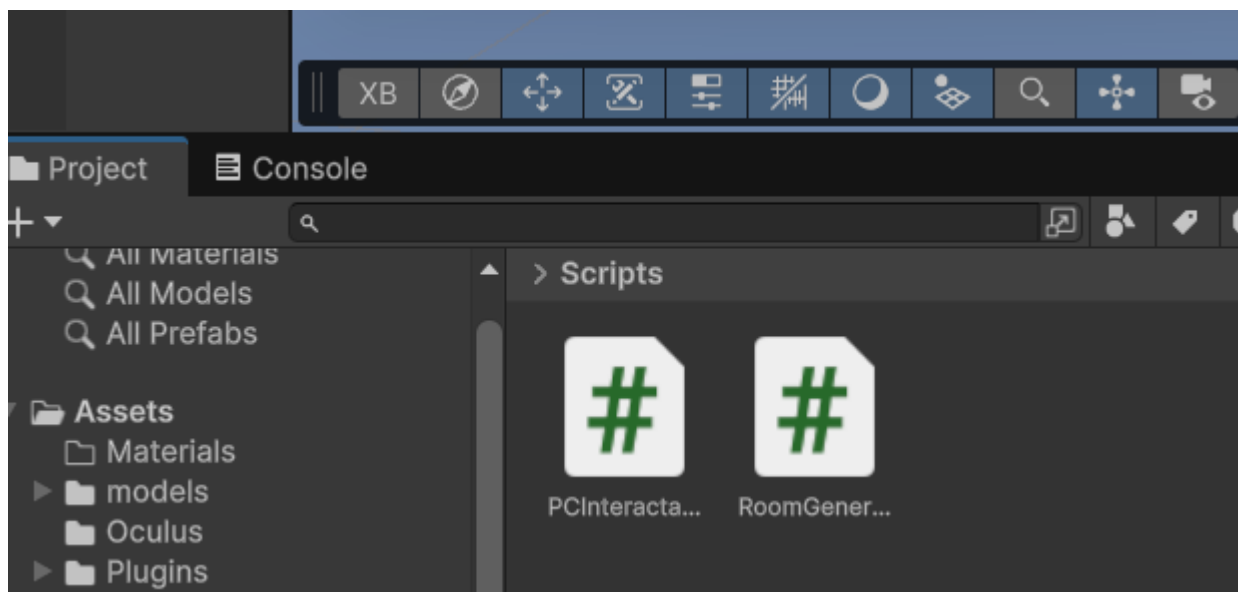
- automatiser le placement des objets dans la salle ;
- ajouter automatiquement certains Components à l'exécution ;
- éviter de reconstruire manuellement toute la scène à chaque modification.

L'automatisation devient particulièrement précieuse lorsque l'on travaille sur plusieurs scènes ou lorsque la salle doit contenir un grand nombre d'objets. Elle garantit également la reproductibilité : deux exécutions successives produisent exactement le même résultat.

8.2. Création des scripts dans Unity

Les scripts ont été créés selon la procédure suivante :

1. Ouverture du dossier Assets/Scripts.
2. Clic droit puis Create → C# Script.
3. Création du script RoomGenerator.
4. Création du script PCInteractable.



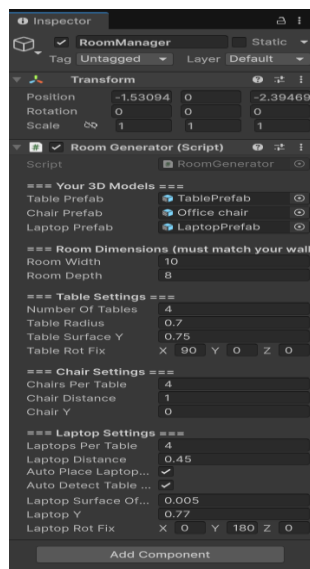
Scripts C# créés dans le dossier Scripts

Le code proprement dit pour RoomGenerator.cs et pour PCInteractable.cs ; Les fichiers complets sont disponibles via le lien Google Drive partagé par les encadrants.

8.3. Ajout du composant RoomGenerator à la scène

Un script C# ne s'exécute pas tout seul : il doit être attaché à un GameObject de la scène pour devenir un Component actif. La procédure a été :

- 1. Création d'un GameObject vide dans la Hierarchy, nommé RoomManager.
- 2. Sélection de ce RoomManager.
- 3. Dans l'Inspector, clic sur Add Component.
- 4. Recherche et ajout du composant RoomGenerator.
- 5. Glisser-déposer des trois Prefabs dans les champs Table Prefab, Chair Prefab et Laptop Prefab.
- 6. Vérification que Room Width et Room Depth correspondent exactement aux dimensions des murs construits précédemment.



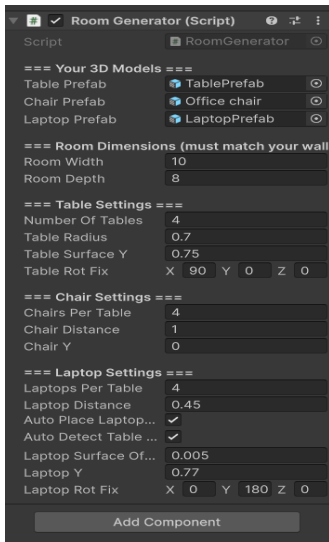
RoomManager avec le composant RoomGenerator et ses champs renseignés

L'usage d'un GameObject vide (RoomManager) comme support du script est une bonne pratique : il ne représente rien visuellement, mais il centralise la logique de gestion de la salle. Cela évite de coupler le script à un objet visible, ce qui faciliterait ultérieurement sa modification ou sa suppression.

8.4. Fonctionnement de RoomGenerator

Le composant RoomGenerator remplit plusieurs rôles essentiels :

- recevoir les Prefabs renseignés dans l'Inspector ;
- connaître les dimensions de la salle via Room Width et Room Depth ;
- calculer des positions appropriées pour les tables ;
- instancier automatiquement les chaises autour de chaque table ;
- poser les ordinateurs portables sur les tables ;
- ajouter certains Components utiles au moment de l'exécution.



Lors de la lecture de la capture de l'Inspector, il est particulièrement important de contrôler les valeurs de Room Width et Room Depth. Si ces valeurs ne correspondent pas aux dimensions réelles de la salle construite avec les Cubes, les objets peuvent apparaître dans les murs ou trop près des bords, ce qui compromet la crédibilité de l'environnement.

Nous avons également ajusté d'autres paramètres exposés dans l'Inspector :

- Number Of Tables pour fixer le nombre de tables (3 dans notre cas) ;
- Table Radius et Table Surface Y pour contrôler la distribution autour des tables et la hauteur de la surface ;
- Chairs Per Table et Chair Distance pour régler la disposition des chaises ;
- Laptops Per Table et Laptop Distance pour configurer les ordinateurs portables ;
- Table Rot Fix, Chair Y, Laptop Rot Fix pour corriger d'éventuels problèmes d'orientation liés aux modèles importés.

Le fait que ces valeurs soient modifiables directement dans l'Inspector, sans toucher au code, illustre bien la séparation entre logique et configuration : le script définit comment les choses se passent, et l'Inspector permet de régler les détails quantitatifs.

8.5. Fonctionnement de PCInteractable

Le script PCInteractable gère le comportement d'un objet interactif dans l'environnement VR. Dans notre projet, il est appliqué à l'ordinateur portable et lui permet de :

- réagir lorsqu'un objet est survolé par la manette ;
- réagir lorsqu'un objet est saisi par l'utilisateur ;
- ajuster son comportement physique au moment du grab (saisie) ou du release (relâchement).

Concrètement, lorsqu'un utilisateur tend la main vers un ordinateur portable et le saisit, le script peut désactiver temporairement la gravité, attacher l'objet à la main, puis rétablir le comportement physique normal lorsqu'il est relâché. Cette logique est essentielle pour produire une interaction fluide et crédible en VR.

Un point méthodologique important : l'Inspector permet de remplir des champs publics déclarés dans le code sans avoir à modifier ce dernier. Le script définit la logique, tandis que l'Inspector fournit les références aux objets et les valeurs numériques. Cette séparation facilite grandement le travail d'équipe, le développeur et le designer pouvant travailler en parallèle.

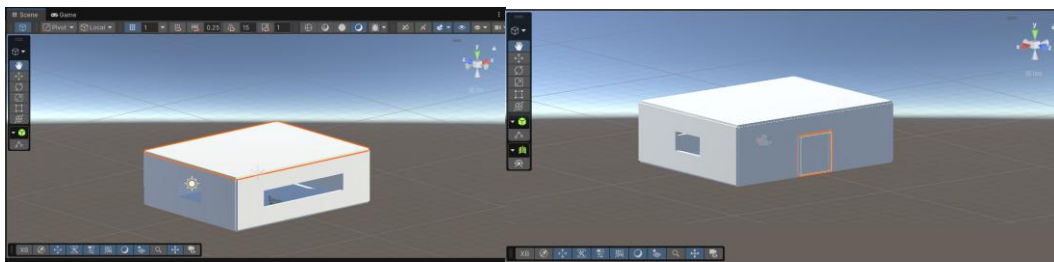
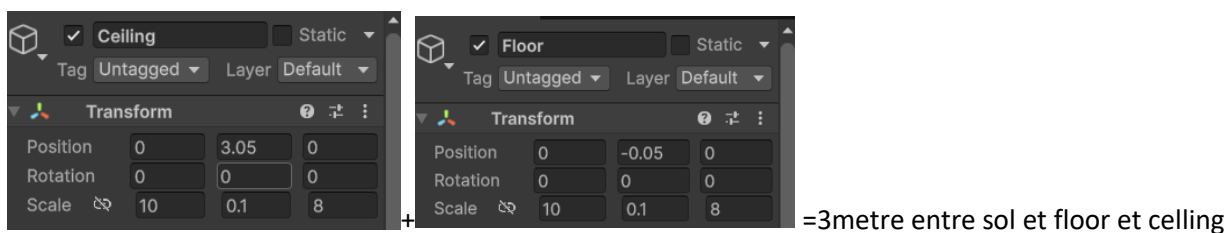
9. Tests progressifs

Le test du projet a été effectué de manière progressive, en trois phases successives, afin d'identifier rapidement la source de tout dysfonctionnement.

9.1. Test 1 : validation de la salle

Avant même de lancer la moindre simulation physique, nous avons contrôlé visuellement la salle construite. Les points vérifiés ont été :

- le sol est parfaitement horizontal ;
- le plafond se trouve à exactement 3 mètres du sol ;
- les quatre murs sont fermés et bien alignés entre eux ;
- l'échelle générale de la pièce est crédible pour un usage VR.

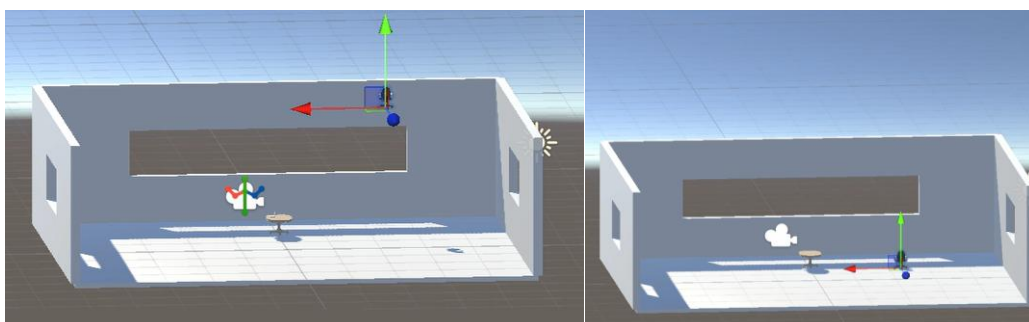


Cette validation visuelle est une étape simple mais essentielle : une erreur de dimensionnement à ce stade se répercute sur toutes les étapes suivantes.

9.2. Test 2 : validation de la chaise

La chaise est l'objet de validation idéal pour la physique. Nous avons observé plusieurs comportements :

- avec la gravité active, la chaise doit tomber naturellement sur le sol au démarrage de la scène ;
- si le sol possède bien un Collider, la chaise s'arrête en entrant en contact avec lui et ne le traverse pas ;
- si le Collider de la chaise est mal choisi ou mal dimensionné, les collisions paraissent incohérentes (la chaise flotte, s'enfonce, ou vibre).



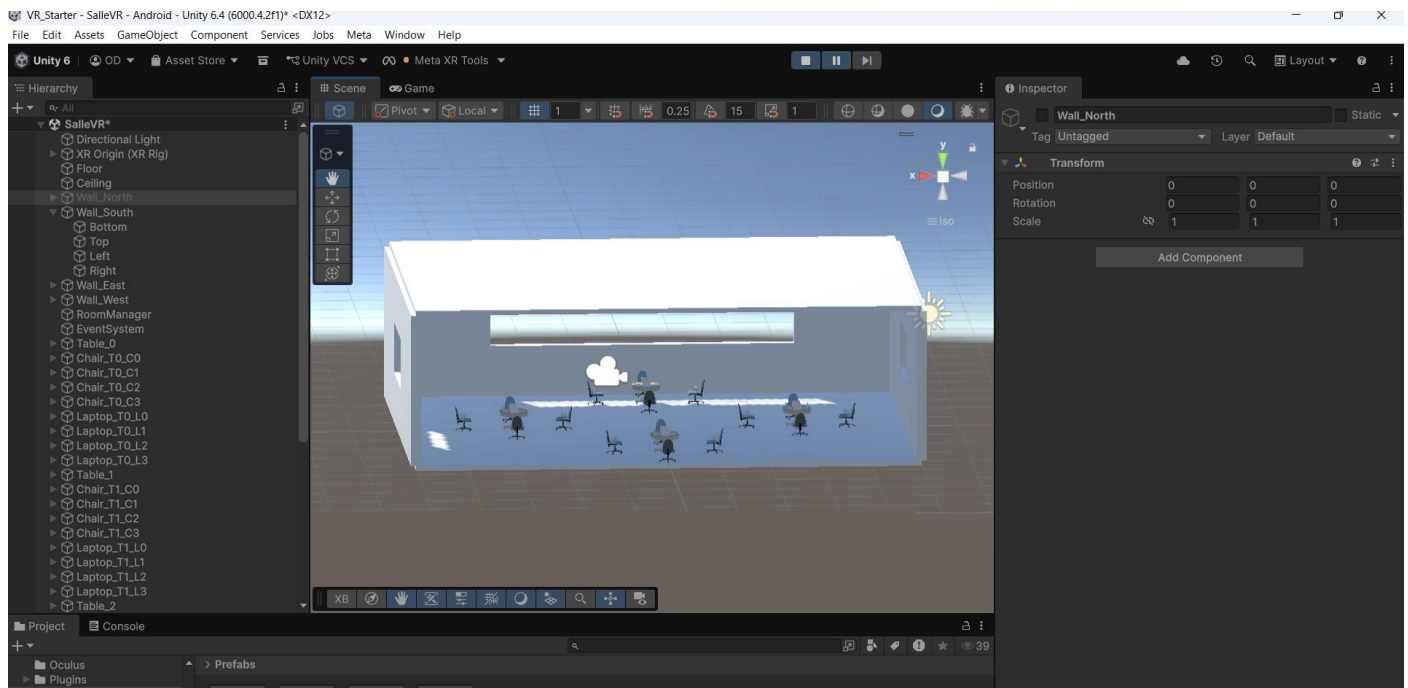
la chaise tombant et s'immobilisant sur le sol (voir video le mouvement pour valider)

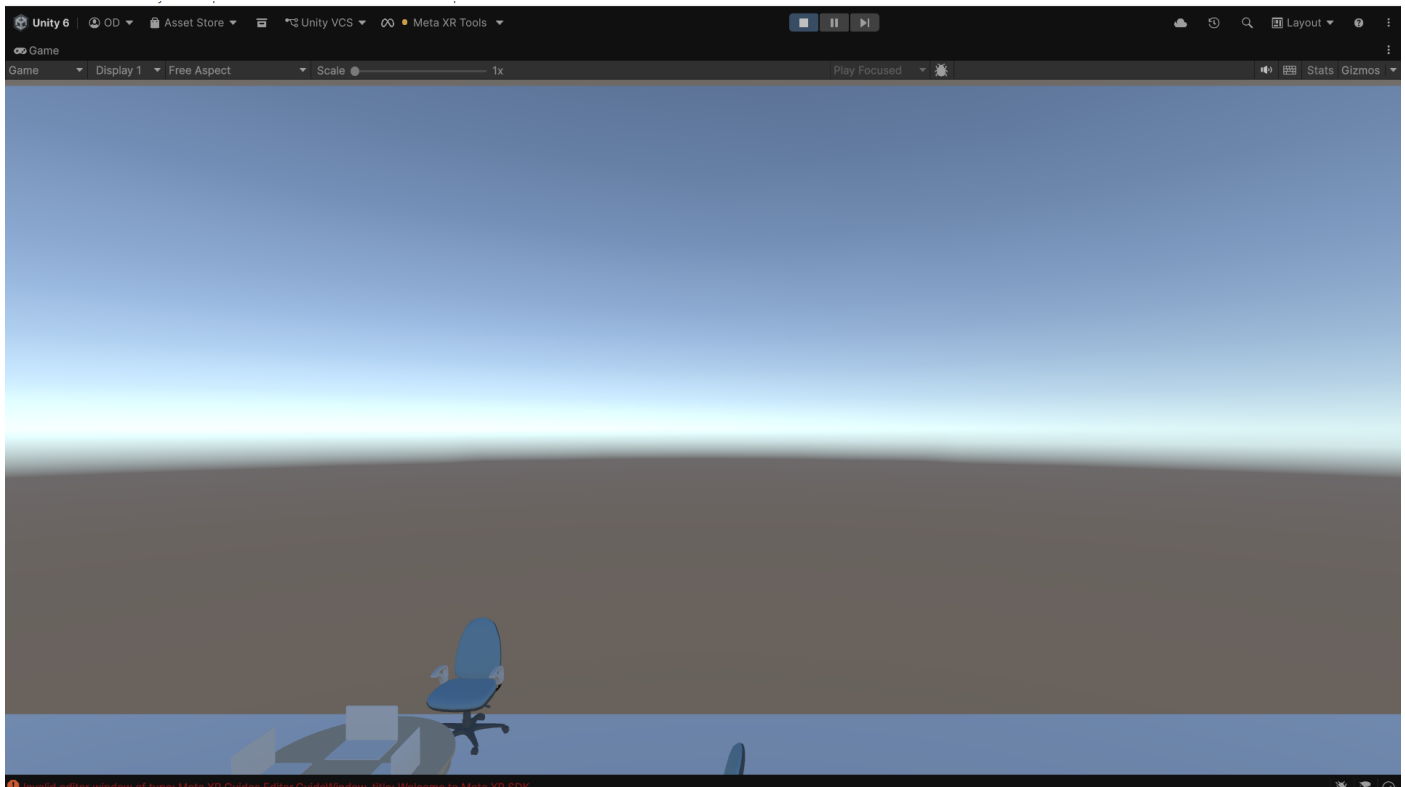
Ce test permet de valider en même temps la configuration du Rigidbody, du Collider de la chaise et du Collider du sol. C'est un test complet et rapide.

9.3. Test 3 : validation du script RoomGenerator

Le dernier test concerne le placement automatique effectué par le script. Lorsque le script fonctionne correctement, on doit observer :

- l'apparition des tables dans la salle ;
- le placement des chaises autour des tables ;
- l'apparition des ordinateurs portables posés sur les tables ;
- l'absence d'objets qui commencent à l'intérieur des murs.





Test 3 : salle meublée par le script RoomGenerator

Erreurs fréquentes rencontrées

Durant nos tests, plusieurs erreurs typiques peuvent se manifester. Nous les listons ici ainsi que leur cause :

- **NullReferenceException** : un Prefab n'a pas été renseigné dans l'Inspector. Le script essaie alors d'instancier un objet qui n'existe pas.
- **Objets mal placés** : les dimensions de la salle dans le script ne correspondent pas à celles des murs.
- **Objets qui tombent à travers le sol** : un Collider est manquant, soit sur le sol, soit sur l'objet qui tombe.
- **Script non actif** : le composant n'a pas été ajouté au RoomManager, ou bien il est décoché dans l'Inspector.

L'identification rapide de ces erreurs tient à la méthode de test progressif : en validant chaque couche indépendamment, nous avons pu isoler efficacement la source des dysfonctionnements.

Problèmes rencontrés et solutions apportées (Génération procédurale)

Lors de la mise en place du script RoomGenerator.cs pour la disposition automatique des meubles, plusieurs problèmes de positionnement et de rotation sont apparus. Ces dysfonctionnements étaient principalement liés aux différences d'échelle (Scale) et aux points de pivot des modèles 3D importés, qui entraient en conflit avec les valeurs codées en dur dans le script initial.

Voici le détail des problèmes rencontrés et les méthodes de résolution appliquées :

1. Problème d'élévation des tables (Flottement et collision avec le sol)

- **Description** : Lors de la génération, les tables n'étaient pas correctement posées sur le sol. Soit elles flottaient en l'air, soit elles passaient à travers le sol (clipping).
- **Cause** : Le code initial plaçait l'axe Y des tables à une hauteur fixe (0.33f). De plus, le modèle 3D de la table possédait une échelle très réduite (0.005), ce qui décalait son point de pivot par rapport à sa surface visuelle.

- **Solution** : Le code a été modifié pour remplacer la valeur fixe par une variable publique (tableSpawnY), rendant ce paramètre accessible directement depuis l'**Inspector** d'Unity. Après plusieurs tests d'ajustement en mode *Play*, la valeur idéale a été trouvée à **0.202**, permettant d'aligner parfaitement les pieds de la table avec le sol.

2. Problème de rotation des chaises

- **Description** : Les chaises générées autour de la table tournaient le dos à celle-ci au lieu d'y faire face.
- **Cause** : L'orientation "Avant" (Forward) du modèle 3D importé de la chaise était inversée par rapport à l'axe logique utilisé par la fonction Quaternion.LookRotation() du script.
- **Solution** : Une correction mathématique a été ajoutée dans le code C#. Une multiplication par un quaternion de compensation (Quaternion.Euler(0f, 180f, 0f)) a été appliquée lors de la génération pour forcer une rotation de 180 degrés sur l'axe Y, orientant ainsi correctement les chaises vers le centre de la table.

3. Problème d'alignement et de positionnement des ordinateurs (Laptops)

- **Description** : Les ordinateurs présentaient trois anomalies : ils flottaient au-dessus de la table, étaient décalés horizontalement, et n'étaient pas orientés en face des chaises (décalage de 90 degrés).
- **Cause** : 1. **La rotation** : La boucle de génération utilisait des calculs d'angles différents pour les chaises et pour les PC. L'ordinateur *i* n'était donc pas assigné à la chaise *i*.

2. **Le flottement** : À cause de la petite taille du modèle (Scale = 0.05), la fonction chargée de calculer la boîte de collision (Bounds) pour coller le PC à la surface de la table calculait mal la base de l'objet.

- **Solution** : Le script a été refactorisé pour que les PC et les chaises utilisent **le même angle de cercle** lors de la génération, garantissant qu'ils soient parfaitement face à face. Ensuite, de nouvelles variables d'ajustement ont été exposées dans l'Inspector pour corriger visuellement les limites du modèle 3D :

Variable Inspector	Fonction
Laptop Distance	Ajustement de la distance par rapport au centre de la table pour rapprocher le PC de la chaise (réglé sur 0.3).
Laptop Angle Offset	Correction finale de la rotation du modèle 3D sur son axe pour faire face à l'utilisateur.
Laptop Extra Offset Y	Ajustement micrométrique de la hauteur pour que la base du PC touche parfaitement la surface du bois (correction du flottement).

Grâce à ces modifications combinant des corrections logiques dans le script C# et des ajustements de paramètres dans l'environnement Unity, la salle se génère désormais de manière réaliste et fonctionnelle.

10. Préparation du build pour Meta Quest 2

La dernière étape du TP consiste à déployer l'application sur le casque Meta Quest 2. Comme le Quest 2 fonctionne sous Android, il est nécessaire de configurer Unity pour générer un fichier compatible. La procédure de build suivie est :

- 1. Ouverture de File → Build Settings.
- 2. Sélection de la plateforme Android.
- 3. Clic sur Switch Platform pour changer de cible de compilation.
- 4. Clic sur Add Open Scenes pour inclure notre scène SalleVR dans le build.
- 5. Branchement du casque Meta Quest 2 en USB à l'ordinateur.
- 6. Clic sur Build And Run pour lancer la compilation et l'installation sur le casque.

Fenêtre Build Settings avec Android sélectionné

Vérifications avant le build

Avant de lancer le build, plusieurs points de contrôle ont été vérifiés :

- OpenXR est activé pour la plateforme Android ;
- la scène a été sauvegardée ;
- aucun message d'erreur (rouge) n'apparaît dans la Console ;
- tous les Prefabs sont correctement renseignés dans le composant RoomGenerator.

Ces vérifications préalables évitent un build défectueux qui aurait pu échouer à l'installation ou au lancement sur le casque.

(reste a tester sur le caque)

11. Conclusion

Ce travail pratique nous a permis de parcourir l'ensemble du cycle de création d'une application de réalité virtuelle, depuis la configuration initiale du projet XR jusqu'au déploiement final sur le casque Meta Quest 2. Cette démarche progressive, alliant construction manuelle puis automatisation logicielle, constitue une méthodologie solide et directement transposable à des projets VR plus complexes.

Au cours de ce développement, nous avons mis en œuvre et assimilé plusieurs piliers fondamentaux de la création sous Unity :

- **L'environnement 3D et les échelles** : La construction d'une **salle** à partir d'objets primitifs nous a démontré l'importance de dimensionner correctement les éléments et de respecter les échelles réelles (1 unité = 1 mètre) en VR.
- **L'optimisation des ressources** : L'utilisation de **Prefabs** s'est révélée indispensable pour maintenir le projet. Configurer un objet une seule fois pour le réutiliser partout garantit un gain de temps et une cohérence globale.
- **La physique et les interactions** : Nous avons manipulé le **Rigidbody**, moteur de la physique et de la gravité, et maîtrisé les **Colliders** (les volumes invisibles gérant les chocs). Cette étape a mis en évidence le compromis technique entre un **Box Collider** (calcul simple et performant, mais géométrie approximative) et un **Mesh Collider** (très fidèle au modèle 3D, mais plus coûteux en ressources).
- **L'automatisation et l'architecture logicielle** : La génération procédurale via des **Scripts C#** nous a permis d'automatiser des tâches répétitives et d'ajouter des composants à la volée. Surtout, elle a illustré le principe de séparation entre le code source (la logique pure) et l'**Inspector** (l'interface permettant d'ajuster les paramètres sans avoir à reprogrammer).

Enfin, la confrontation avec le matériel réel lors du build sur le Meta Quest 2 a confirmé qu'une application VR fonctionnelle repose avant tout sur la rigueur des vérifications préalables : une configuration OpenXR valide, des Prefabs correctement assignés et une Console exempte d'erreurs sont les conditions *sine qua non* pour la réussite du déploiement.

13. Annexes

Les fichiers de code sont fournis séparément afin de conserver ce rapport centré sur la méthode et sur les manipulations Unity. Les ressources disponibles sont :

- **Annexe A** : Annexe_A_RoomGenerator.cs — script de génération automatique de la salle meublée.

```
using UnityEngine;
.
.
.
public class RoomGenerator : MonoBehaviour
{
    [Header("=== Your 3D Models ===")]
    public GameObject tablePrefab;    // drag your table .obj here
    public GameObject chairPrefab;    // drag your chair .obj here
    public GameObject laptopPrefab;   // drag your laptop .obj here
    .
    [Header("=== Room Dimensions (must match your walls) ===")]
    public float roomWidth = 10f;    // X axis (Wall_East to Wall_West)
    public float roomDepth = 8f;    // Z axis (Wall_North to Wall_South)
    .
    [Header("=== Table Settings ===")]
    public int numberOfTables = 4;
    public float tableRadius = 0.7f; // half the table diameter
    public float tableSurfaceY = 0.75f; // height of tabletop in meters
    public Vector3 tableRotFix = new Vector3(90f, 0f, 0f); // fix sideways import
    .
    // FIX 1 : Y de spawn de la table.
    // Mets 0f si le pivot du modele est a la base.
    // Si la table s'enfonce dans le sol, augmente (ex: 0.05f).
    // Si elle flotte encore, diminue (ex: -0.05f).
    [Header("=== Table Spawn Y (ajuste selon ton modele) ===")]
    public float tableSpawnY = 0f;
    .
    [Header("=== Chair Settings ===")]
    public int chairsPerTable = 4;
    public float chairDistance = 1.0f; // how far chairs sit from table center
    public float chairY = 0f; // chairs sit on the floor
    .
    // FIX 2 : correction de rotation des chaises.
    // 180f = chaise face a la table. Change si toujours retournee.
    public Vector3 chairRotFix = new Vector3(0f, 180f, 0f);
    .
    [Header("=== Laptop Settings ===")]
    public int laptopsPerTable = 4;
    public float laptopDistance = 0.45f; // how far laptops sit from table center
    public bool autoPlaceLaptopOnTable = true;
    public bool autoDetectTableSurface = true;
    public float laptopSurfaceOffset = 0.005f; // small gap to avoid z-fighting
    public float laptopY = 0.77f; // fallback if auto placement is disabled
    public Vector3 laptopRotFix = new Vector3(0f, 180f, 0f); // fix imported laptop forward
axis
    .
    void Start()
    {
        PlaceEverything();
    }
    .
    void PlaceEverything()
    {
        float margin = tableRadius + 0.5f;
```

```

float halfW = roomWidth / 2f - margin;
float halfD = roomDepth / 2f - margin;

Vector3[] tablePositions = GetTablePositions(numberOfTables, halfW, halfD);

for (int t = 0; t < tablePositions.Length; t++)
{
    Vector3 tablePos = tablePositions[t];

    // ---- Spawn TABLE ----
    GameObject table = Instantiate(tablePrefab);
    table.name = "Table_" + t;

    // FIX 1 applique : on utilise tableSpawnY au lieu de 0.33f code en dur
    table.transform.position = new Vector3(tablePos.x, tableSpawnY, tablePos.z);
    table.transform.rotation = Quaternion.Euler(tableRotFix);
    SetupCollider(table, false);

    // FIX 3 : on attend la fin du frame pour que les bounds soient corrects
    // apres rotation. GetObjectTopY lit les bounds world-space, c est OK.
    float tableTopY = GetObjectTopY(table, tableSurfaceY);

    // ---- Spawn CHAIRS around the table ----
    for (int c = 0; c < chairsPerTable; c++)
    {
        float angle = (360f / chairsPerTable) * c * Mathf.Deg2Rad;
        Vector3 chairPos = new Vector3(
            tablePos.x + Mathf.Sin(angle) * chairDistance,
            chairY,
            tablePos.z + Mathf.Cos(angle) * chairDistance
        );

        Vector3 lookDir = new Vector3(tablePos.x, 0f, tablePos.z) - chairPos;

        // FIX 2 applique : on ajoute chairRotFix (180 par default)
        // pour que la chaise fasse face a la table et non l inverse
        Quaternion chairRot = Quaternion.LookRotation(lookDir)
            * Quaternion.Euler(chairRotFix);

        GameObject chair = Instantiate(chairPrefab);
        chair.name = $"Chair_T{t}_C{c}";
        chair.transform.position = chairPos;
        chair.transform.rotation = chairRot;
        SetupCollider(chair, true);
        MakeGrabbable(chair, false);
    }

    // ---- Spawn LAPTOPS on the table ----
    for (int l = 0; l < laptopsPerTable; l++)
    {
        float angle = (360f / laptopsPerTable) * l * Mathf.Deg2Rad;
        Vector3 outwardDir = new Vector3(
            Mathf.Sin(angle),
            0f,
            Mathf.Cos(angle)
        );

        // FIX 3 applique : tableTopY est maintenant correct car la table
        // est a la bonne hauteur grace au Fix 1
        Vector3 laptopPos = new Vector3(
            tablePos.x + outwardDir.x * laptopDistance,
            autoPlaceLaptopOnTable ? tableTopY : laptopY,
            tablePos.z + outwardDir.z * laptopDistance
        );

        Quaternion laptopRot =
            Quaternion.LookRotation(outwardDir, Vector3.up) *
            Quaternion.Euler(laptopRotFix);

        GameObject laptop = Instantiate(laptopPrefab);
        laptop.name = $"Laptop_T{t}_L{l}";
    }
}

```

```

        laptop.transform.rotation = laptopRot;
        laptop.transform.position = laptopPos;

        if (autoPlaceLaptopOnTable)
            SnapObjectBottomToY(laptop, tableTopY + laptopSurfaceOffset);

        SetupCollider(laptop, true);
        MakeGrabbable(laptop, true);
    }
}

Vector3[] GetTablePositions(int count, float halfW, float halfD)
{
    Vector3[] positions = new Vector3[count];

    if (count == 1)
    {
        positions[0] = Vector3.zero;
    }
    else
    {
        float radius = Mathf.Min(halfW, halfD);

        for (int i = 0; i < count; i++)
        {
            float angle = (360f / count) * i * Mathf.Deg2Rad;
            positions[i] = new Vector3(
                Mathf.Sin(angle) * radius,
                0f,
                Mathf.Cos(angle) * radius
            );
        }

        return positions;
    }
}

float GetObjectTopY(GameObject obj, float fallbackY)
{
    if (autoDetectTableSurface && TryGetObjectBounds(obj, out Bounds bounds))
        return bounds.max.y;

    return fallbackY;
}

void SnapObjectBottomToY(GameObject obj, float targetY)
{
    if (!TryGetObjectBounds(obj, out Bounds bounds))
    {
        Vector3 fallbackPos = obj.transform.position;
        fallbackPos.y = targetY;
        obj.transform.position = fallbackPos;
        return;
    }

    Vector3 pos = obj.transform.position;
    pos.y += targetY - bounds.min.y;
    obj.transform.position = pos;
}

bool TryGetObjectBounds(GameObject obj, out Bounds bounds)
{
    Renderer[] renderers = obj.GetComponentsInChildren<Renderer>();
    if (renderers.Length == 0)
    {
        bounds = default;
        return false;
    }

    bounds = renderers[0].bounds;
}

```

```

    for (int i = 1; i < renderers.Length; i++)
        bounds.Encapsulate(renderers[i].bounds);

    return true;
}

void SetupCollider(GameObject obj, bool isGrabbable)
{
    MeshRenderer[] renderers = obj.GetComponentsInChildren<MeshRenderer>();
    foreach (MeshRenderer mr in renderers)
    {
        GameObject part = mr.gameObject;

        if (part.GetComponent<Collider>() == null)
        {
            MeshCollider mc = part.AddComponent<MeshCollider>();

            if (isGrabbable)
                mc.convex = true;
        }

        Rigidbody rb = obj.GetComponent<Rigidbody>();
        if (rb == null)
            rb = obj.AddComponent<Rigidbody>();

        if (isGrabbable)
        {
            rb.mass = 1f;
            rb.useGravity = true;
        }
        else
        {
            rb.isKinematic = true;
            rb.useGravity = false;
        }
    }
}

void MakeGrabbable(GameObject obj, bool keepStillUntilGrab)
{
    UnityEngine.XR.Interaction.Toolkit.Interactables.XRGrabInteractable grab =
obj.GetComponent<UnityEngine.XR.Interaction.Toolkit.Interactables.XRGrabInteractable>();
    if (grab == null)
        grab =
obj.AddComponent<UnityEngine.XR.Interaction.Toolkit.Interactables.XRGrabInteractable>();

    grab.movementType =
UnityEngine.XR.Interaction.Toolkit.Interactables.XRBaseInteractable.MovementType.VelocityTra
cking;
    grab.throwOnDetach = true;

    PCInteractable interact = obj.GetComponent<PCInteractable>();
    if (interact == null)
        interact = obj.AddComponent<PCInteractable>();

    interact.normalScale = obj.transform.localScale;
    interact.hoverScaleMultiplier = 1.08f;
    interact.grabbedScale = obj.transform.localScale * 1.12f;
    interact.keepStillUntilGrab = keepStillUntilGrab;
    interact.useGravityAfterRelease = true;
    interact.RefreshPhysicsState();
}
}

```

- **Annexe B : Annexe_B_PCInteractable.cs** — script d'interaction VR appliqué à l'ordinateur portable.

using UnityEngine;

```
using UnityEngine.XR.Interaction.Toolkit;
using UnityEngine.XR.Interaction.Toolkit.Interactors;
```

```
[RequireComponent(typeof(UnityEngine.XR.Interaction.Toolkit.Interactables.XRGrabInteractable))]
```

```
public class PCInteractable : MonoBehaviour
```

```
{
```

```
    [Header("Sizes")]
```

```
    public Vector3 normalScale = Vector3.one;
```

```
    public Vector3 grabbedScale = Vector3.zero;
```

```
    public float hoverScaleMultiplier = 1.08f;
```

```
    [Header("Speed of size change")]
```

```
    public float scaleSpeed = 8f;
```

```
    [Header("Haptics")]
```

```
    [Range(0f, 1f)] public float hoverHapticAmplitude = 0.2f;
```

```
    public float hoverHapticDuration = 0.05f;
```

```
    [Range(0f, 1f)] public float grabHapticAmplitude = 0.35f;
```

```
    public float grabHapticDuration = 0.08f;
```

```
    [Header("Physics")]
```

```
    public bool keepStillUntilGrab = false;
```

```
    public bool useGravityAfterRelease = true;
```

```
    private UnityEngine.XR.Interaction.Toolkit.Interactables.XRGrabInteractable _grab;
```

```
    private Rigidbody _rb;
```

```
    private Vector3 _targetScale;
```

```
    private bool _isGrabbed = false;
```

```
    void Awake()
```

```
{
```

```
        _grab = GetComponent<UnityEngine.XR.Interaction.Toolkit.Interactables.XRGrabInteractable>();
```

```
        _rb = GetComponent<Rigidbody>();
```

```
        // Use actual object scale if not set manually
```

```
        if (normalScale == Vector3.one && transform.localScale != Vector3.one)
```

```

        normalScale = transform.localScale;

        if (grabbedScale == Vector3.zero)
            grabbedScale = normalScale * 1.12f;

        _targetScale = normalScale;

        RefreshPhysicsState();
    }

    void OnEnable()
    {
        _grab.hoverEntered.AddListener(OnHoverEnter);
        _grab.hoverExited.AddListener(OnHoverExit);
        _grab.selectEntered.AddListener(OnGrab);
        _grab.selectExited.AddListener(OnRelease);
    }

    void OnDisable()
    {
        _grab.hoverEntered.RemoveListener(OnHoverEnter);
        _grab.hoverExited.RemoveListener(OnHoverExit);
        _grab.selectEntered.RemoveListener(OnGrab);
        _grab.selectExited.RemoveListener(OnRelease);
    }

    void OnHoverEnter(HoverEnterEventArgs args)
    {
        if (!_isGrabbed)
            _targetScale = normalScale * hoverScaleMultiplier;

        SendHaptics(args.interactorObject, hoverHapticAmplitude, hoverHapticDuration);
    }

    void OnHoverExit(HoverExitEventArgs args)
    {

```

```

    if (!_isGrabbed) _targetScale = normalScale;
}

void OnGrab(SelectEnterEventArgs args)
{
    _isGrabbed = true;
    _targetScale = grabbedScale;
    SendHaptics(args.interactorObject, grabHapticAmplitude, grabHapticDuration);

    if (_rb != null)
    {
        _rb.isKinematic = false;
        _rb.useGravity = false;
    }
}

void OnRelease(SelectExitEventArgs args)
{
    _isGrabbed = false;
    _targetScale = normalScale; // returns to full size on drop

    if (_rb != null)
    {
        _rb.isKinematic = false;
        _rb.useGravity = useGravityAfterRelease;
    }
}

void Update()
{
    transform.localScale = Vector3.Lerp(
        transform.localScale, _targetScale, Time.deltaTime * scaleSpeed
    );
}

public void RefreshPhysicsState()

```

```

{
    if (_rb == null || !_isGrabbed)
        return;

    if (keepStillUntilGrab)
    {
        _rb.isKinematic = true;
        _rb.useGravity = false;
        return;
    }

    _rb.isKinematic = false;
    _rb.useGravity = useGravityAfterRelease;
}

void SendHaptics(IXRInteractor interactor, float amplitude, float duration)
{
    if (interactor == null || amplitude <= 0f || duration <= 0f)
        return;

    if (interactor is XRBaseInputInteractor inputInteractor)
        inputInteractor.SendHapticImpulse(amplitude, duration);
}
}

```

```

• /*
• * Copyright (c) Meta Platforms, Inc. and affiliates.
• * All rights reserved.
• *
• * Licensed under the Oculus SDK License Agreement (the "License");
• * you may not use the Oculus SDK except in compliance with the License,
• * which is provided at the time of installation or download, or which
• * otherwise accompanies this software in either electronic or hard copy form.
• *
• * You may obtain a copy of the License at
• *
• * https://developer.oculus.com/licenses/oculussdk/
• *
• * Unless required by applicable law or agreed to in writing, the Oculus SDK
• * distributed under the License is distributed on an "AS IS" BASIS,
• * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
• * See the License for the specific language governing permissions and
• * limitations under the License.
• */
•
• using System.Collections;

```


- `using System.Collections.Generic;`
- `using UnityEngine;`
-
- `public class ColliderSizeConstraint : MonoBehaviour`
- `{`
- `public Vector3 size;`
- `public int expandingAxis;`
- `public Transform pointA;`
- `public Transform pointB;`
- `public float wideSideOffset;`
-
- `private void Update()`
- `{`
- `var wideSideLength = Vector3.Distance(pointA.position, pointB.position);`
- `wideSideLength -= wideSideOffset;`
-
- `var worldScale = transform.parent.lossyScale;`
-
- `var desiredSize = size;`
- `desiredSize[expandingAxis] = wideSideLength;`
- `var newSize = new Vector3(desiredSize.x / worldScale.x, desiredSize.y /`
- `worldScale.y, desiredSize.z / worldScale.z);`
- `transform.localScale = newSize;`
- `}`
- `}`
-

- **Ressources en ligne** : lien Google Drive partagé par les encadrants pour consulter les codes sources et télécharger les fichiers .obj des modèles 3D.

https://drive.google.com/file/d/1J_sMx1U0J1QG0j4-mI-V2uCggMd2qs/view

Il est important de rappeler que le code ne dispense pas de comprendre la structure de la scène. Avant toute exécution des scripts, la salle, les Prefabs et les Components visibles dans l'Inspector doivent être vérifiés avec soin.